HELPHIVE

Complete Beginner Code Guide

From first file to production request - with architecture, authentication, database, deployment, and a practical reading plan.

Built from the actual repository
Snapshot: main branch, commit 3476e5b. The guide reflects members-only access, separate registration and sign-in, JWT security, post ownership, Neon PostgreSQL, and Render hosting.

Prepared for a reader with no previous Java, Spring Boot, Angular, API, or database knowledge.

Generated: August 2026

Contents

Use the chapters in order on your first reading. Later, use the file reference and flow diagrams as a map while you work in the code.

1. Start here: what this project is 3

2. Vocabulary for a complete beginner 5

3. Technology stack: what is used and why 6

4. The exact order to read the files 7

5. Root files, dependencies, and configuration 8

6. Frontend startup and the Angular mental model 9

7. Angular forms, template, and UI logic 10

8. Frontend services and HTTP 11

9. Spring Boot startup and dependency injection 12

10. Backend layers: controller, service, repository, entity 13

11. Database design and JPA 14

12. Registration from start to finish 16

13. Login and JWT creation 17

14. Every authenticated request 18

15. Member-only access, ownership, and privacy 19

16. Post CRUD and business logic 20

17. Validation, errors, and response DTOs 21

18. Run and understand it locally on Windows 22

19. Production build: Docker, Render, and Neon 23

20. GitHub Actions, tests, and verification 24

21. Debugging map 25

22. A seven-day learning plan 26

23. Safe exercises that make the code yours 27

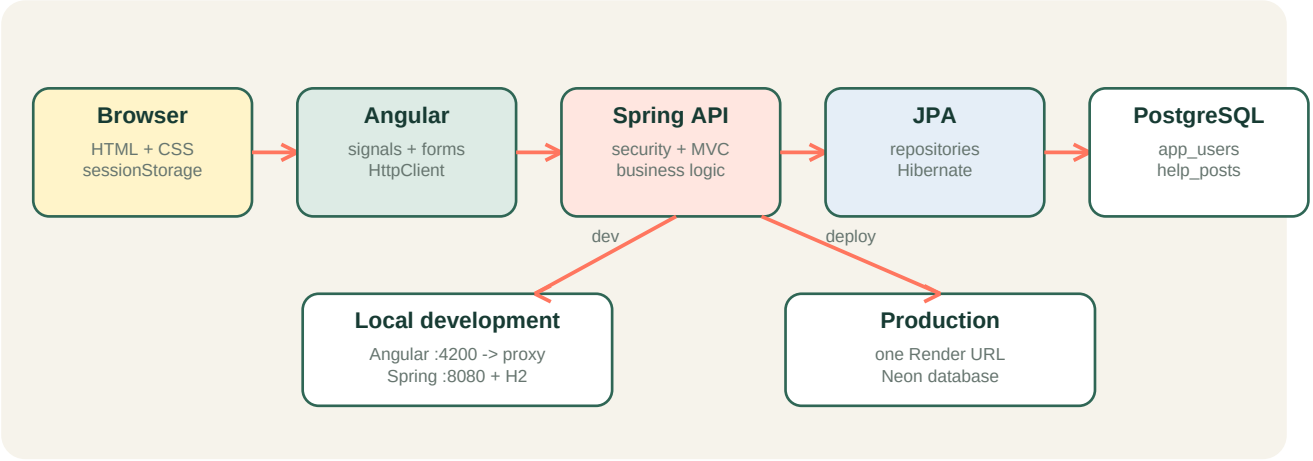
24. Complete source-file reference 28

25. API quick reference 31

26. Final mental model and glossary 33

1. Start here: what this project is

HelpHive is a members-only community exchange. A person creates an account, signs in, reads requests or offers, publishes a post, contacts other members, and moves their own post from OPEN to IN_PROGRESS to COMPLETED.



The five parts to keep in your head

Part	Plain-English job	Where it lives
`Angular frontend`	Draws the screen, validates forms, remembers UI state, and calls the API.	frontend/src
`Spring Boot backend`	Protects data, applies rules, and exposes JSON endpoints.	backend/src/main/java
`Database`	Permanently stores users and posts in production.	Neon PostgreSQL
`Delivery`	Builds one deployable container and hosts it for free.	Dockerfile + render.yaml
`Quality`	Runs automated checks whenever code is pushed.	.github/workflows/ci.yml

The central idea
The browser never talks directly to the database. Angular talks to Spring Boot using HTTP. Spring Boot decides what is allowed and uses JPA to talk to PostgreSQL.

What happens before and after sign-in

Visitor state	What Angular shows	What the API permits
`No account / signed out`	Only the non-dismissible account and sign-in gate.	Register, login, and health check only.
`Account created`	A success message and sign-in form.	Still no post access until login.
`Signed in`	The complete community interface.	All post reads; create; owner-only changes.
`Expired token`	Returns to sign-in with a session-expired message.	401 Unauthorized.

2. Vocabulary for a complete beginner

You do not need to memorize these definitions. Read them once, then return here whenever a word feels unfamiliar.

Word	Meaning in this project
`Frontend`	The Angular application running in the user's browser.
`Backend`	The Spring Boot Java application running on Render or localhost:8080.
`API`	A set of URLs such as POST /api/auth/login that accept and return JSON.
`HTTP method`	The action word: GET reads, POST creates, PUT replaces, PATCH changes part, DELETE removes.
`JSON`	Text format used to move structured data between Angular and Spring.
`Entity`	A Java object mapped to a database table.
`Repository`	A Spring interface that performs database operations.
`Service`	A class containing business rules and orchestration.
`Controller`	The class that receives an HTTP request and returns an HTTP response.
`Dependency injection`	Spring or Angular creates an object and supplies its dependencies automatically.
`JWT`	A signed token proving which user logged in. It is not the password.
`Hash`	One-way password transformation. BCrypt hashes cannot be reversed into passwords.
`DTO / record`	A safe, shaped object used for API input or output instead of exposing entities.
`Signal`	Angular state that automatically causes the template to update when its value changes.
`Observable`	An RxJS stream representing an asynchronous result such as an HTTP response.
`Environment variable`	A deployment-time setting such as DATABASE_URL or JWT_SECRET.

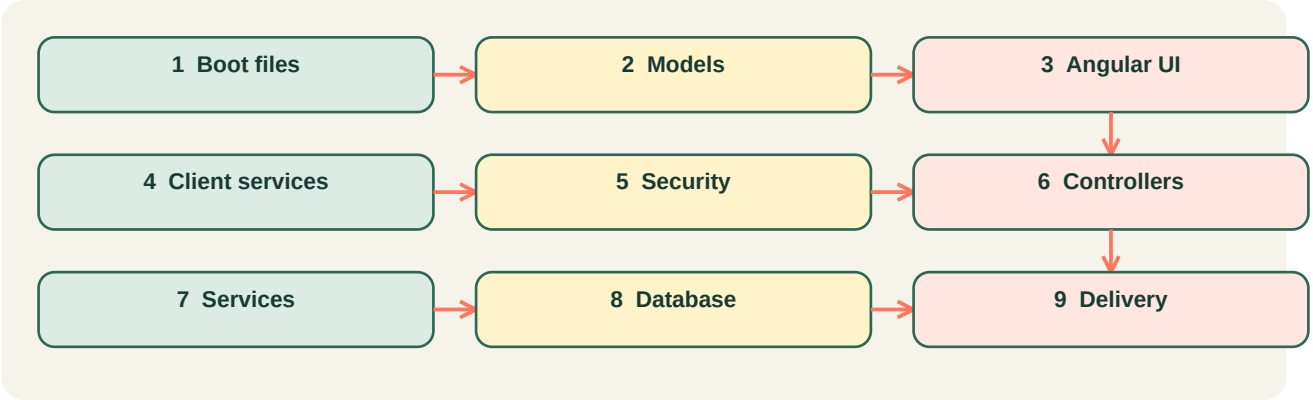
3. Technology stack: what is used and why

Technology	Role	Why it fits HelpHive
`Java 21`	Backend language	Strong types, mature ecosystem, long-term support, and common enterprise usage.
`Spring Boot 4.1`	Backend framework	Starts an HTTP server, wires objects, loads configuration, and integrates the Spring libraries.
`Spring Web MVC`	REST API	Maps HTTP methods and paths to controller methods.
`Spring Security`	Authentication boundary	Blocks member endpoints and attaches the JWT user to each request.
`Bean Validation`	Input safety	Annotations such as @NotBlank and @Size reject bad requests consistently.
`Spring Data JPA`	Data access	Creates repository implementations and maps Java entities to SQL tables.
`Hibernate`	JPA implementation	Generates SQL and tracks entity changes inside transactions.
`Angular 22`	Browser application	Component templates, reactive forms, signals, and production bundling.
`TypeScript 6`	Frontend language	Adds compile-time types to JavaScript and documents API shapes.
`RxJS`	Async programming	HttpClient returns Observables; subscribe handles success and failure.
`SCSS`	Styling	Keeps the custom responsive interface in maintainable CSS-compatible syntax.
`H2`	Local database	Runs in memory with no setup, ideal for learning and tests.
`PostgreSQL`	Production database	Durable relational data, constraints, joins, and indexes.
`Neon`	Hosted PostgreSQL	Provides the external free database used by Render.
`Maven Wrapper`	Java build	Downloads the correct Maven version and builds/tests the backend.
`npm + Angular CLI`	Frontend build	Installs exact packages and runs development, tests, and production builds.
`Docker`	Production packaging	Builds Angular and Java, then runs one small Java container.
`Render Blueprint`	Hosting configuration	Creates/configures the web service from render.yaml.
`GitHub Actions`	Continuous integration	Tests backend and frontend on pushes and pull requests.

Why Angular and Spring are separate during development
Angular's development server gives fast browser refresh at port 4200. Spring runs the API at port 8080. proxy.conf.json forwards /api calls so the frontend can use one relative URL.

4. The exact order to read the files

Do not begin by reading every file alphabetically. Follow the application's movement from startup to UI to API to database.



1. Read **README.md** for the product goal and commands.
2. Read **frontend/package.json**, **backend/pom.xml**, and **application.yml** to see dependencies and runtime settings.
3. Follow browser startup: **index.html -> main.ts -> app.config.ts -> app.ts -> app.html**.
4. Read TypeScript contracts: **auth.model.ts** and **help-post.model.ts**.
5. Read browser communication: **auth.service.ts**, **help-post.service.ts**, and **auth.interceptor.ts**.
6. Follow Java startup: **HelpHiveApplication.java -> SecurityConfig.java -> JwtAuthenticationFilter.java**.
7. Follow accounts: **AuthController -> AuthService -> AppUserRepository -> AppUser**, with **JwtService** during login.
8. Follow posts: **HelpPostController -> HelpPostService -> HelpPostRepository -> HelpPost**.
9. Read input/output records and **GlobalExceptionHandler** to understand validation and errors.
10. Finish with **Dockerfile**, **render.yaml**, and **ci.yml** to understand delivery.

> **Best reading technique**
> Pick one user action and keep asking: Which method runs next? What data crosses the boundary? Which class owns the rule? Start with login, then list posts, then create a post.

Feature tracing formula

```
HTML click or form submit
  -> method in app.ts
  -> Angular service method
  -> auth interceptor adds Bearer token
  -> Spring Security validates token
  -> controller endpoint
  -> service business rule
  -> repository / entity / database
  -> response DTO becomes JSON
  -> Angular subscribe updates a signal
  -> app.html automatically re-renders
```

5. Root files, dependencies, and configuration

README.md

The human entry point. It explains features, architecture, commands, APIs, deployment, and future ideas. Read it before code because it tells you what behavior the code is supposed to produce.

backend/pom.xml

Maven's project file. The Spring Boot parent manages compatible dependency versions. Dependencies add web, JPA, validation, actuator, security, H2, PostgreSQL, and tests. The Spring Boot Maven plugin packages an executable JAR.

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

frontend/package.json

npm's project manifest. dependencies are required in the running application. devDependencies support building and testing. package-lock.json freezes exact transitive versions so local builds, CI, and Docker are reproducible.

Command	Meaning
`npm start`	Angular development server with the /api proxy.
`npm run build`	Optimized production browser bundle.
`npm test -- --watch=false`	One non-interactive unit-test run.
`./mvnw verify`	Compile, run backend tests, and package the JAR.

backend/src/main/resources/application.yml

Spring configuration uses environment variables first and safe development defaults second. Locally, no DATABASE_URL means H2. On Render, the Neon values override H2.

```
datasource:
  url: ${DATABASE_URL:jdbc:h2:mem:helphive...}
app:
  security:
    jwt-secret: ${JWT_SECRET:local-development-secret...}
    jwt-expiration: ${JWT_EXPIRATION:PT2H}
```

Configuration rule
Never put a real Neon password or production JWT secret in Git. Render stores those values as environment variables.

6. Frontend startup and the Angular mental model

Startup chain

File	What happens
`src/index.html`	Browser shell with . It does not contain the community UI.
`src/main.ts`	Calls bootstrapApplication(App, appConfig). This creates the root component.
`app.config.ts`	Registers HttpClient, the auth interceptor, router, and global error listeners.
`app.routes.ts`	Empty because this is currently a single-page, single-component application.
`app.ts`	Owns UI state, forms, actions, and calls to frontend services.
`app.html`	Declarative template. It shows the auth gate or community based on signals.
`app.scss`	Component layout, colors, modals, responsive rules, and auth-gate design.
`styles.scss`	Global fonts, color variables, reset rules, and accessibility helper.

What a component is

The **App** class is the brain for the root screen. The **@Component** metadata connects the class to app.html and app.scss. Angular evaluates template expressions against the class.

```
@Component({
  selector: 'app-root',
  imports: [ReactiveFormsModule, DatePipe],
  templateUrl: './app.html',
  styleUrl: './app.scss',
})
export class App { ... }
```

Signals: state that redraws the view

A signal is a value Angular watches. **posts.set(newPosts)** changes the value; **posts()** reads it. The template's @if and @for blocks re-evaluate automatically.

```
readonly posts = signal<HelpPost[]>([]);
readonly loading = signal(true);
readonly resultLabel = computed(
  () => `${this.posts().length} matches`
);
```

7. Angular forms, template, and UI logic

Reactive forms

FormBuilder creates three typed forms: postForm, loginForm, and registerForm. Validators run in the browser for fast feedback. The backend validates again because browser validation can be bypassed.

Form	Important validation	Submit method
`registerForm`	Name required; valid email; password 8-72 characters.	submitAuth() registration branch
`loginForm`	Valid email and a password value.	submitAuth() login branch
`postForm`	Title, 20-800 character description, location, category, type.	submit()

Members-only rendering

The outer template condition is the first UI gate. When authenticated() is false, Angular does not create the header, community board, post cards, or footer. It creates only the auth gate.

```
@if (auth.authenticated()) {
  <!-- complete community UI -->
}

@if (!auth.authenticated() || authModalOpen()) {
  <!-- account and sign-in gate -->
}
```

UI protection is not enough
A person can call an API without using your Angular screen. That is why SecurityConfig also requires authentication for /api/**. The backend is the real security boundary.

How filtering works

Search, category, type, and status are signals. Each control changes a signal and calls refresh(). HelpPostService converts non-empty values into HTTP query parameters. The server filters the returned posts.

8. Frontend services and HTTP

auth.model.ts and help-post.model.ts

These interfaces are TypeScript contracts. They document the JSON shape expected from Spring. They do not exist at runtime, but the compiler catches mistakes while you code.

AuthService

AuthService is provided once for the whole application. Registration calls the API but does not save a session. Login saves the JWT and safe user object. Logout removes both.

```
login(input: LoginInput): Observable<AuthResponse> {
  return this.http.post<AuthResponse>('/api/auth/login', input)
    .pipe(tap(response => this.save(response)));
}
```

The token and user use **sessionStorage**. They survive page refreshes in the same tab session but are removed when that browser session ends. authenticated is computed from both stored pieces.

authInterceptor

The interceptor is a central checkpoint for outgoing API calls. It clones each /api request and adds the Authorization header when a token exists. Login and registration are sent without a token when signed out.

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

HelpPostService

This service contains only HTTP details: paths, methods, query parameters, and response types. It does not decide ownership. Security-sensitive rules live on the server.

9. Spring Boot startup and dependency injection

HelpHiveApplication.java

Java starts in main(). SpringApplication.run launches the embedded server, scans the com.helphive.api package, creates components, configures the database, and registers endpoints.

```
@SpringBootApplication(
  exclude = UserDetailsServiceAutoConfiguration.class
)
public class HelpHiveApplication {
  public static void main(String[] args) {
    SpringApplication.run(HelpHiveApplication.class, args);
  }
}
```

The exclusion prevents Spring from creating an unrelated default development user and password because HelpHive provides its own JWT-based authentication.

Dependency injection

Classes declare required collaborators in their constructors. Spring creates one suitable object and passes it in. For example, AuthService receives AppUserRepository, PasswordEncoder, and JwtService. This makes responsibilities visible and classes testable.

```
public AuthService(AppUserRepository repository,
                   PasswordEncoder passwordEncoder,
                   JwtService jwtService) {
  this.repository = repository;
  this.passwordEncoder = passwordEncoder;
  this.jwtService = jwtService;
}
```

Common annotations

Annotation	Meaning
`@RestController`	Class receives HTTP requests and serializes return values as JSON.
`@Service`	Business-logic component managed by Spring.
`@Entity`	Class maps to a database table.
`@Configuration`	Class supplies application configuration or beans.
`@Bean`	Method return value becomes a managed Spring object.
`@Transactional`	Database work runs as one unit; failures roll back.
`@Valid`	Run Bean Validation on the request body.
`@AuthenticationPrincipal`	Inject the user established by the JWT filter.

10. Backend layers: controller, service, repository, entity

The post feature is intentionally layered. Each layer has one type of responsibility.

Layer	HelpHive class	Responsibility
`Controller`	HelpPostController	HTTP paths, methods, parameters, status codes, request/response shapes.
`Service`	HelpPostService	Filtering, ownership, state changes, and transaction boundaries.
`Repository`	HelpPostRepository	Database CRUD and generated count queries.
`Entity`	HelpPost	Fields, table mapping, relationships, and timestamps.
`DTO`	HelpPostRequest/Response	Validated external input and safe external output.

Why not put everything in the controller?

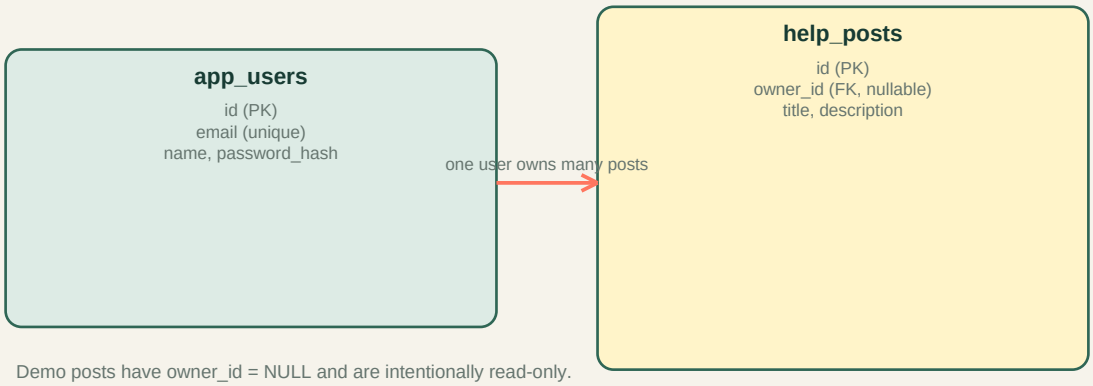
A controller should translate HTTP into a use case. If database and business logic are mixed there, code becomes harder to test and reuse. Services also provide a clear place for authorization rules such as requireOwner().

Repository magic

JpaRepository already supplies save, findById, findAll, count, and delete. Spring reads method names such as countByStatus and generates SQL at runtime.

```
public interface HelpPostRepository
    extends JpaRepository<HelpPost, Long> {
  long countByStatus(PostStatus status);
  long countByType(PostType type);
}
```

11. Database design and JPA



AppUser entity

@Entity maps AppUser to app_users. Email has both an application duplicate check and a unique database constraint. passwordHash stores BCrypt output, never the original password. UserRole is saved as readable text such as MEMBER.

HelpPost entity

HelpPost maps to help_posts. @ManyToOne means many posts may point to one owner. The owner relation is nullable only so the original seeded demo posts can remain. New posts always receive the signed-in AppUser as owner.

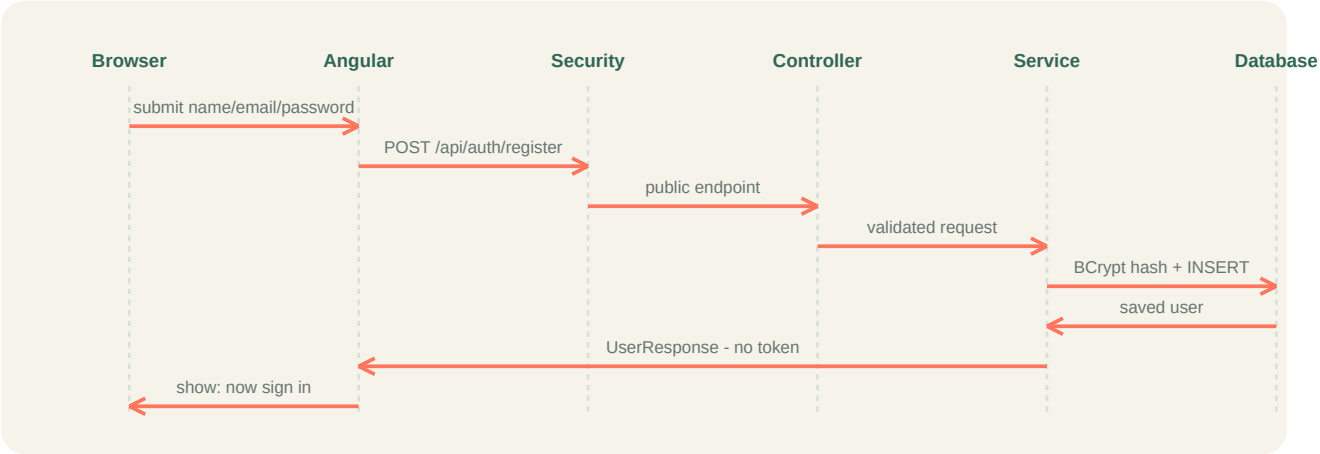
Database column	Source	Rule
`title`	HelpPostRequest	Required, maximum 90.
`description`	HelpPostRequest	Required, 20-800.
`author_name`	Signed-in AppUser.name	Client cannot impersonate another author.
`contact`	Signed-in AppUser.email	Client cannot submit a different contact.
`owner_id`	JWT user -> AppUser	Used for edit/delete/status authorization.
`status`	Default OPEN	Moves to IN_PROGRESS or COMPLETED.
`created_at / updated_at`	JPA lifecycle callbacks	Set automatically.

H2 versus Neon PostgreSQL

Environment	Database	Persistence
`Local normal run`	In-memory H2	Cleared when the backend process stops.
`Automated tests`	Separate in-memory H2	Created and dropped for the test context.
`Hosted Render app`	Neon PostgreSQL	Survives Render sleep and redeploys.

Prototype choice
hibernate.ddl-auto=update is convenient for this learning project. Mature production systems usually use versioned migrations such as Flyway so schema changes are reviewed and repeatable.

12. Registration from start to finish



Exact code path

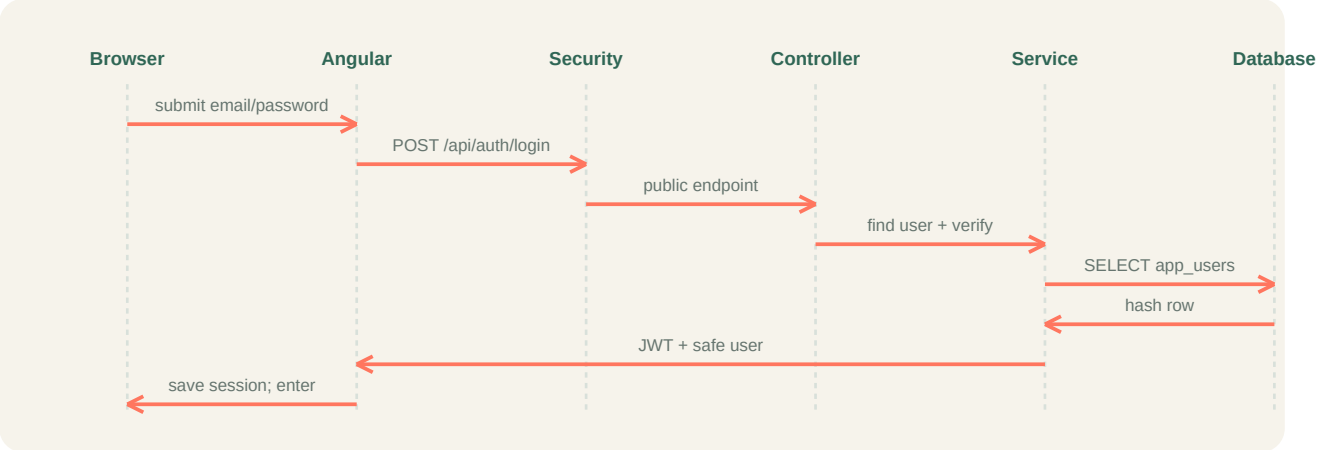
1. app.html submits registerForm to **App.submitAuth()**.
2. Angular validators reject missing name, bad email, or a password outside 8-72 characters.
3. AuthService sends **POST /api/auth/register** with JSON.
4. SecurityConfig permits this endpoint without a JWT.
5. AuthController validates RegisterRequest and calls AuthService.register().
6. AuthService trims and lowercases email, rejects duplicates, and BCrypt-hashes the password.
7. AppUserRepository.save() inserts into app_users inside a transaction.
8. The API returns UserResponse only - no password hash and no JWT.
9. Angular switches to the sign-in tab and pre-fills the email. Community access is still locked.

```
POST /api/auth/register
{
  "name": "Alex Kim",
  "email": "alex@example.com",
  "password": "strong-pass-123"
}

201 Created
{
  "id": 7,
  "name": "Alex Kim",
  "email": "alex@example.com",
  "role": "MEMBER"
}
```

> **Why registration does not log in**
> The project deliberately requires two steps: create the account, then prove the credentials through the login endpoint before receiving access.

13. Login and JWT creation



Password verification

The database stores a salted BCrypt hash. passwordEncoder.matches(rawPassword, storedHash) performs the check. The raw password is not saved and cannot be recovered from the hash.

JWT contents

JwtService creates a compact three-part token using HMAC-SHA256 (HS256). The header and payload are Base64URL encoded. The signature is calculated with JWT_SECRET.

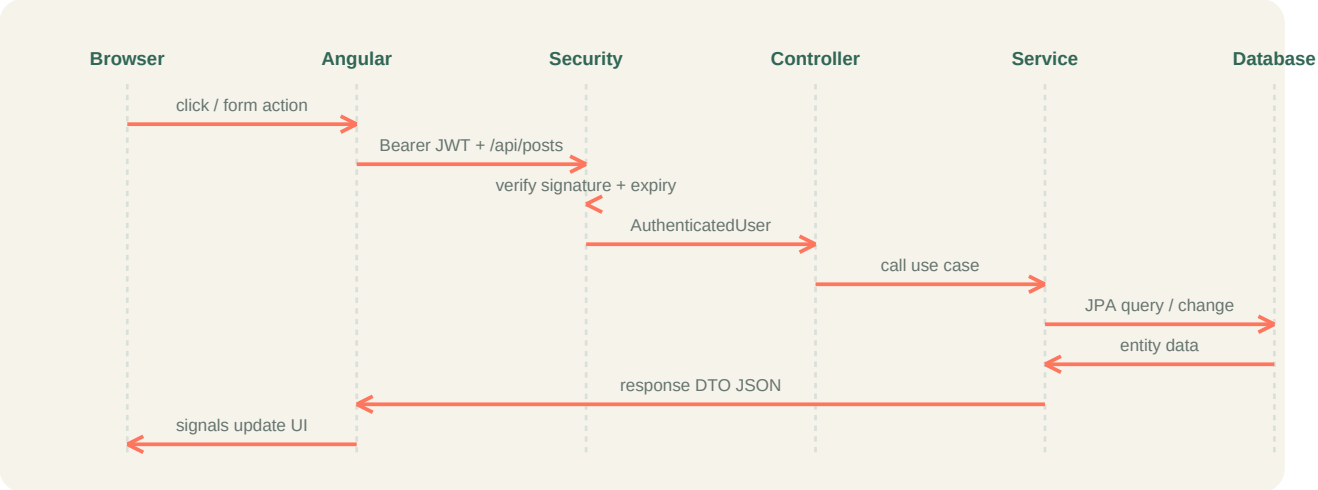
Claim	Meaning
`sub`	Subject: normalized email.
`uid`	Database user ID.
`name`	Display name.
`role`	MEMBER or ADMIN.
`iat`	Issued-at Unix time.
`exp`	Expiration Unix time; two hours by default.

```
POST /api/auth/login
{ "email": "alex@example.com", "password": "strong-pass-123" }

200 OK
{
  "token": "eyJ...signed-token...",
  "user": { "id": 7, "name": "Alex Kim",
            "email": "alex@example.com", "role": "MEMBER" }
}
```

JWT is signed, not encrypted
A JWT payload can be decoded by its holder. Never put passwords or secrets inside it. The signature prevents undetected changes; HTTPS protects it while travelling over the network.

14. Every authenticated request



JwtAuthenticationFilter

This OncePerRequestFilter runs once for each request. It reads the Authorization header, removes 'Bearer ', validates the signature and expiration through JwtService, creates an AuthenticatedUser, and stores authentication in Spring's SecurityContext.

```
String header = request.getHeader(HttpHeaders.AUTHORIZATION);
AuthenticatedUser user = jwtService.parse(header.substring(7));
SecurityContextHolder.getContext().setAuthentication(authentication);
```

SecurityConfig rules

```
register, login, health  -> permitAll()
/api/**                  -> authenticated()
static Angular files     -> permitAll()
```

If no valid authentication exists for a protected API request, Spring returns JSON with HTTP 401. If a signed-in member tries to alter another member's post, the service throws AccessDeniedException and returns HTTP 403.

401 versus 403

Status	Meaning	Example
`401 Unauthorized`	Identity missing or invalid.	No token, malformed token, or expired token.
`403 Forbidden`	Identity is known but action is not allowed.	Member deletes another member's post.

15. Member-only access, ownership, and privacy

Defense in depth

Layer	Protection
`Angular template`	Does not render community UI while signed out.
`App.refresh()`	Does not request posts or stats while signed out; clears data on logout/401.
`Interceptor`	Adds JWT to API requests.
`Spring Security`	Rejects every /api/** request except register/login without authentication.
`HelpPostService`	Checks owner ID before update, status change, or delete.
`Database`	Unique email and foreign-key relationship protect integrity.

Ownership is based on server identity

A post request does not include authorName, contact, or ownerId. During creation, HelpPostService loads the AppUser identified by the signed JWT and copies the user's name and email. This prevents a client from claiming to be someone else.

```
AppUser owner = requireUser(currentUser.id());
post.setOwner(owner);
post.setAuthorName(owner.getName());
post.setContact(owner.getEmail());
```

Owner check

requireOwner compares help_posts.owner_id with the authenticated user's ID. The UI also hides owner buttons, but the server comparison is what provides security.

```
if (post.getOwner() == null ||
    !post.getOwner().getId().equals(currentUser.id())) {
  throw new AccessDeniedException(
    "Only the post owner can make this change");
}
```

Demo-post exception
Seeded posts have no AppUser owner. They are visible to members but remain read-only because requireOwner rejects owner == null.

16. Post CRUD and business logic

User action	HTTP request	Controller -> service
`Load/filter board`	GET /api/posts?search=...	findAll -> findAll
`Open one post`	GET /api/posts/{id}	findById -> findById
`Load counters`	GET /api/posts/stats	stats -> stats
`Create`	POST /api/posts	create -> create
`Edit details`	PUT /api/posts/{id}	update -> update + requireOwner
`Advance status`	PATCH /api/posts/{id}/status	updateStatus -> requireOwner
`Delete`	DELETE /api/posts/{id}	delete -> requireOwner

Create post flow

1. User completes postForm; Angular validates it.
2. HelpPostService on the frontend sends the input JSON with the JWT added by the interceptor.
3. Spring Security establishes currentUser.
4. HelpPostController validates HelpPostRequest.
5. The backend service copies allowed fields, attaches the real owner identity, and saves.
6. The controller returns 201 Created and a Location header.
7. Angular closes the modal, displays a toast, reloads posts and statistics, and the template updates.

Filtering implementation

The current implementation loads all posts sorted newest-first and filters in Java streams. That is easy to understand and fine for a small portfolio dataset. For thousands of posts, move filters and pagination into repository queries so PostgreSQL does the work.

Status state machine

```
OPEN  ->  IN_PROGRESS  ->  COMPLETED
```

The UI advances only forward. The API currently accepts any valid PostStatus value, so stricter transition validation would be a useful future exercise.

17. Validation, errors, and response DTOs

Two validation layers

Angular validation improves usability. Jakarta Bean Validation protects the server. Never depend only on frontend validation because any HTTP client can bypass the browser UI.

Exception / condition	HTTP status	Handled by
`Invalid request fields`	400	GlobalExceptionHandler.handleValidation
`Bad email/password`	401	handleBadCredentials
`Duplicate account`	409	handleConflict
`Post/user not found`	404	handleNotFound
`Not the post owner`	403	handleAccessDenied
`Unexpected exception`	500	Generic safe error; internal details are not exposed.

Why request and response records exist

Entities represent stored data. API records represent allowed external data. RegisterRequest accepts a password, but UserResponse never returns passwordHash. HelpPostRequest omits author and owner fields, while HelpPostResponse adds computed ownedByCurrentUser.

> **A subtle privacy point**
> HelpPostResponse hides contact when currentUserId is null. With the current members-only API, a valid user is always present for post calls. Keeping this output rule adds another layer of protection if public reads are reintroduced later.

18. Run and understand it locally on Windows

Terminal 1 - backend

```
cd /c/Users/gujju/Documents/Codex/2026-08-13/i/outputs/github-projects/helphive/backend
cmd.exe /c mvnw.cmd spring-boot:run
```

Wait for 'Started HelpHiveApplication'. Spring runs at http://localhost:8080 and uses in-memory H2 unless database environment variables are set.

Terminal 2 - frontend

```
cd /c/Users/gujju/Documents/Codex/2026-08-13/i/outputs/github-projects/helphive/frontend
npm install
npm start
```

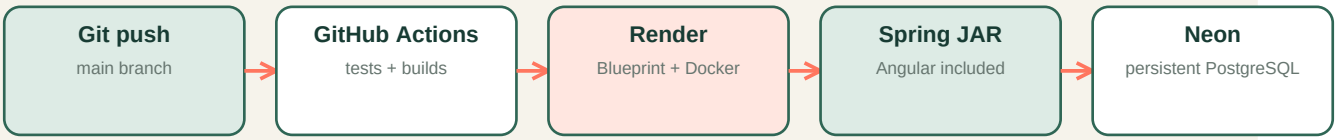
Open http://localhost:4200. proxy.conf.json forwards /api to port 8080. Accounts created locally disappear after the backend stops because H2 is in memory.

A deliberate learning session

1. Open browser Developer Tools -> Network.
2. Create an account and inspect POST /api/auth/register. Notice there is no token.
3. Sign in and inspect POST /api/auth/login. Find token and user in the response.
4. Open Application -> Session Storage and find helphive_token and helphive_user.
5. Load the board. Inspect GET /api/posts and its Authorization header.
6. Create a post. Compare the request JSON with the response JSON; owner fields come from the server.
7. Sign out. Confirm the community disappears and protected API calls return 401 without the token.

> **Do not paste tokens publicly**
> A JWT is temporary login authority. Treat it like a password while it is valid.

19. Production build: Docker, Render, and Neon



Render runs one container. Spring serves both the Angular files and the REST API.

Dockerfile stages

Stage	Work
`1. frontend-build`	Node 24 runs npm ci and ng build.
`2. backend-build`	Maven downloads dependencies, copies Angular browser files into Spring static resources, and packages the JAR.
`3. runtime`	Small Java 21 JRE image runs as a non-root spring user.

The result is one process and one public URL. Spring serves index.html, hashed JavaScript/CSS files, and /api endpoints. SpaForwardController forwards non-file browser routes to index.html.

render.yaml

Setting	Purpose
`runtime: docker`	Build using the repository Dockerfile.
`plan: free`	Use Render's free web-service instance.
`healthCheckPath`	Render checks /actuator/health.
`autoDeployTrigger: commit`	Deploy after commits to the connected branch.
`DATABASE_* sync:false`	Secret Neon connection values entered in Render.
`JWT_SECRET generateValue:true`	Render generates and stores a random signing secret.
`JAVA_TOOL_OPTIONS`	Limits Java heap as a percentage of container memory.

20. GitHub Actions, tests, and verification

Continuous integration

.github/workflows/ci.yml runs on pushes to main and pull requests. The backend job installs Java 21 and runs Maven verify. The frontend job installs Node 24, runs npm ci, unit tests, and the production build.

Backend integration test

HelpPostControllerTest starts the Spring context with a test H2 database and uses MockMvc to make HTTP-like requests without opening a real network port. Its four scenarios cover CRUD, validation, authentication, ownership, registration, login, and duplicate accounts.

Test concept	Why it matters
`Anonymous GET/POST -> 401`	Proves community data is actually members-only.
`Register then login`	Proves the two-step access requirement.
`Owner versus other member`	Proves 403 ownership enforcement.
`Invalid fields`	Proves server validation produces useful errors.
`CRUD and status`	Proves normal feature behavior end to end.

Frontend tests

app.spec.ts verifies application creation, branding, and the anonymous account gate. auth.service.spec.ts verifies that registration does not create a session and logout removes session data. Vitest and jsdom provide the test environment.

What the build checks do not prove

Passing tests cannot prove the absence of every bug or security issue. Manual browser testing, dependency updates, rate limiting, monitoring, backups, and security review still matter.

21. Debugging map

Symptom	Start here	Likely area
`Blank page`	Browser Console	Angular compile/runtime error or missing bundle.
`Sign-in rejected`	Network response + Render logs	Credentials, user row, BCrypt match, auth error.
`401 after login`	Request Authorization header	Missing/expired token, JWT secret changed.
`403 on edit`	Response body + owner IDs	Signed-in user does not own the post or demo owner is null.
`Posts do not load`	GET /api/posts in Network	Backend sleeping, 401, database connection, server error.
`Local Angular API error`	proxy.conf.json + backend terminal	Backend not running on port 8080.
`Render deploy fails`	Deploy logs from first error	Docker build, Java/Node compile, missing environment variable.
`No users in Neon`	Branch/database selection	Wrong branch, deployment not live, or account created only in local H2.
`Data disappears locally`	application.yml datasource	Expected H2 in-memory behavior.

Read errors from the bottom up

In Java stack traces, find the first 'Caused by' related to com.helphive. In Angular, read the first red error and its file/line. In Render, find the earliest failing build command instead of only the final 'deploy failed' message.

22. A seven-day learning plan

Day	Read	Do
1	Chapters 1-5; startup/config files	Run both servers and identify the two ports.
2	app.ts, app.html, models	Change one label and add one category icon.
3	Angular services/interceptor	Inspect requests in browser Network tools.
4	Spring controllers and DTOs	Add a harmless GET endpoint or response field.
5	Services, repositories, entities	Trace create post from request to database row.
6	Security, registration, login, JWT	Decode a test JWT payload and identify its claims.
7	Tests, Docker, Render	Run checks, commit a small change, and observe CI/deploy.

Use breakpoints instead of only reading

In a Java IDE, place breakpoints in AuthController.login, AuthService.login, JwtService.issue, JwtAuthenticationFilter, HelpPostController.findAll, and HelpPostService.create. In browser tools, use Network and Sources. Watching values move is faster than guessing.

23. Safe exercises that make the code yours

Exercise	Files to change	What you learn
`Add a FOOD category`	Category.java, help-post.model.ts, app.ts	Keep Java and TypeScript contracts synchronized.
`Add post urgency`	Entity, request/response, model, form/template	Full-stack field lifecycle.
`Add My Posts filter`	App state, client query, service/repository query	Identity-aware filtering.
`Validate status transitions`	HelpPostService + tests	Business rules and test-driven changes.
`Add pagination`	Repository/service/controller + Angular	Scalable database querying.
`Add email verification`	User fields, token/mail service, auth UI	Account lifecycle and security.
`Move JWT to HttpOnly cookie`	Security config, controller, Angular auth	Stronger browser token protection.
`Add Flyway migrations`	pom.xml + db/migration files	Production schema management.

Change one vertical slice at a time
For a new field, update database/entity, backend DTO/service, frontend interface/form/template, tests, then documentation. Build after each small step.

24. Complete source-file reference

This catalog tells you why each important file exists. Use it after following the reading order, not as the first learning step.

Root and delivery

File	Purpose
`README.md`	Project overview, API, local commands, deployment, and future work.
`Dockerfile`	Three-stage production build and non-root Java runtime.
`render.yaml`	Render service, health check, auto-deploy, database secrets, JWT secret.
`.github/workflows/ci.yml`	Backend and frontend CI jobs.
`.gitignore / .dockerignore`	Prevent generated, local, and unnecessary files from being committed or copied.
`LICENSE`	MIT permission terms.

Frontend

File	Purpose
`package.json / package-lock.json`	Scripts, direct packages, and frozen dependency tree.
`angular.json`	Angular build, serve, test, assets, styles, and bundle budgets.
`proxy.conf.json`	Local /api forwarding to Spring port 8080.
`src/index.html`	Browser document and app-root mount point.
`src/main.ts`	Bootstraps the standalone App component.
`src/styles.scss`	Global design tokens and base element rules.
`app.config.ts`	Registers HttpClient/interceptor/router.
`app.routes.ts`	Currently empty route list.
`app.ts`	Root UI state, forms, user actions, API subscriptions.
`app.html`	Auth gate, community page, filters, cards, and modals.
`app.scss`	All root component visual and responsive styling.
`auth.model.ts`	Account input/output TypeScript interfaces.
`auth.service.ts`	Register, login, logout, token/user session storage.
`auth.interceptor.ts`	Adds JWT Authorization header to API calls.
`help-post.model.ts`	Post enums, DTO interfaces, statistics shape.
`help-post.service.ts`	Post HTTP methods and filter parameters.
`*.spec.ts`	Angular unit tests.

Backend

File / group	Purpose
`HelpHiveApplication.java`	Main method and Spring Boot startup.
`application.yml`	Database, port, actuator, CORS, JWT configuration.
`auth/*Request, *Response`	Validated auth inputs and safe outputs.
`AuthController.java`	Register, login, and current-user endpoints.
`AuthService.java`	Account creation, email normalization, BCrypt verification, JWT issue.
`JwtService.java`	HS256 token creation, signature verification, expiration parsing.
`AuthenticatedUser.java`	Identity stored in Spring SecurityContext.
`AppUser.java`	app_users entity.
`AppUserRepository.java`	User lookup and duplicate check.
`UserRole.java`	MEMBER and future ADMIN values.
`SecurityConfig.java`	Stateless rules, BCrypt bean, 401/403 JSON handlers.
`JwtAuthenticationFilter.java`	Turns Bearer token into authenticated request identity.
`HelpPostController.java`	REST endpoints for posts and statistics.
`HelpPostService.java`	Filtering, CRUD, owner rules, statistics.
`HelpPostRepository.java`	Post persistence and count queries.
`HelpPost.java`	help_posts entity and user relationship.
`HelpPostRequest/Response`	Validated post input and safe/computed output.
`Category/PostType/PostStatus`	Allowed enum values.
`StatusUpdateRequest/PostStats`	Small endpoint-specific records.
`GlobalExceptionHandler.java`	Consistent API errors.
`ApiError + custom exceptions`	Error response shape and named failure types.
`DemoDataConfig.java`	Adds six ownerless posts to an empty non-test database.
`WebConfig.java`	Development CORS rules.
`SpaForwardController.java`	Forwards browser routes to Angular index.html.
`HelpPostControllerTest.java`	Spring integration/security tests.
`test application.yml`	Isolated create-drop H2 database for tests.

25. API quick reference

Method	Path	Access	Success
POST	/api/auth/register	Public	201 + UserResponse
POST	/api/auth/login	Public	200 + JWT/user
GET	/api/auth/me	Member	200 + user
GET	/api/posts	Member	200 + post array
GET	/api/posts/{id}	Member	200 + post
GET	/api/posts/stats	Member	200 + counts
POST	/api/posts	Member	201 + created post
PUT	/api/posts/{id}	Owner	200 + updated post
PATCH	/api/posts/{id}/status	Owner	200 + updated post
DELETE	/api/posts/{id}	Owner	204 no content
GET	/actuator/health	Public	200 when healthy

Useful Neon queries

```
-- Safe user list: deliberately excludes password_hash
SELECT id, name, email, role, created_at
FROM app_users
ORDER BY created_at DESC;

-- Posts with owner identity
SELECT p.id, p.title, p.status, u.email AS owner_email
FROM help_posts p
LEFT JOIN app_users u ON u.id = p.owner_id
ORDER BY p.created_at DESC;
```

26. Final mental model and glossary

When you feel lost, return to this single sentence:

> **The complete system in one sentence**
> Angular turns user actions into authenticated HTTP requests; Spring Security identifies the member; controllers receive the request; services apply rules; repositories persist entities in PostgreSQL; DTOs return safe JSON; signals redraw the Angular interface.

Compact glossary

Term	Remember this
`Component`	Angular screen logic plus its template and style.
`Signal`	Watched UI state.
`Observable`	Future asynchronous values from HttpClient.
`Interceptor`	Central modifier for outgoing HTTP requests.
`Endpoint`	One HTTP method and path handled by a controller.
`Bean`	Object created and managed by Spring.
`Transaction`	Database work that succeeds or rolls back as a unit.
`Entity`	Java object mapped to a table row.
`Repository`	Data-access interface.
`DTO`	Purpose-built API data shape.
`JWT`	Signed, expiring identity token.
`401`	Sign in is missing or invalid.
`403`	Signed in, but action is not permitted.
`CORS`	Browser rule for requests between different origins.
`Environment variable`	Secret or setting supplied outside source code.

You now know where to begin

1. Open app.ts and follow submitAuth() through AuthService to AuthController and AuthService in Java.
2. Then follow refresh() through HelpPostService to HelpPostController and HelpPostService.
3. Finally follow submit() through create(), the owner assignment, repository.save(), and the returned response.

Do not try to remember every line. Learn the path data follows, the responsibility of each layer, and the security rule enforced at each boundary. The individual syntax becomes much easier after the architecture is familiar.